

Securing a Serverless Inventory API with AWS WAF

Technical Blog Report: API Gateway Protection, IP Filtering, Rate Limiting, SQL Injection Defense, and CloudWatch Monitoring

AWS WAF | API Gateway | Lambda | Amazon RDS | CloudWatch

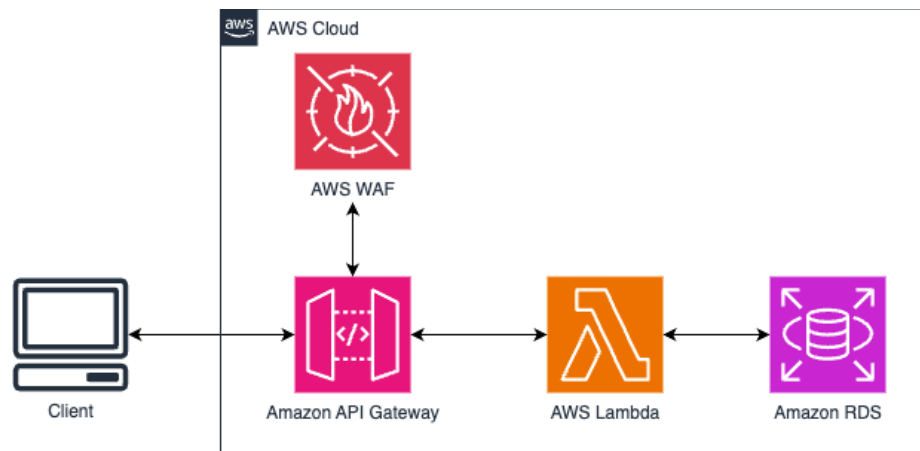


Figure 1 - High-level architecture: external client requests are inspected by AWS WAF before reaching API Gateway, Lambda, and the backend RDS database.

Table of Contents

- 1. Introduction
- 2. Architecture Overview
- 3. Threat Model
- 4. Security Controls Implemented
- 5. Implementation Walkthrough
- 6. Attack Validation and Detection Results
- 7. Rule Evaluation Model
- 8. Monitoring and Operational Visibility
- 9. Security Lessons Learned

1. Introduction

This post documents the implementation of AWS WAF as a protective layer in front of a REST API exposed through Amazon API Gateway. The API returns real-time inventory information from a backend MySQL database and is intended to be consumed only by selected partners and resellers. Because the API exposes sensitive product and business data, access must be restricted, abuse must be controlled, and malicious request patterns must be monitored.

The implementation follows a default-deny security model. A Web ACL is associated with the API Gateway stage, trusted partner traffic is identified through an IP set, excessive requests are blocked with a rate-based rule, and SQL injection payloads are detected using an AWS managed rule group. The result is a layered API protection model that combines access control, request classification, abuse prevention, and operational visibility.

2. Architecture Overview

The application architecture is serverless and API-driven. The data path is intentionally simple:

```
External Client
-> AWS WAF Web ACL
-> Amazon API Gateway REST API
-> AWS Lambda proxy function
-> Amazon RDS MySQL inventory table
```

AWS WAF is placed at the API entry point. Each HTTP request is evaluated against the Web ACL before it can reach API Gateway. If the request is allowed, API Gateway invokes the Lambda function, which queries the RDS database and returns the inventory response. If the request is blocked, the client receives a forbidden response and the backend is never reached.

3. Threat Model

The implementation focuses on four practical API security risks.

Threat	Attack Behavior	Business Risk	AWS WAF Defense
Unauthorized API access	Requests from non-partner IP addresses	Inventory data exposure	Default block + IP set allow logic
API abuse / over-requesting	High request volume from a single client	Lambda concurrency exhaustion and service degradation	Rate-based rule
SQL injection	SQL syntax injected into query parameters	Disclosure of profit, supplier or warehouse data	AWSManagedRulesSQLiRuleSet
Untrusted geography	Requests from locations outside the expected market	Expanded attack surface and policy bypass attempts	Geo match rule as an additional control

The key design decision is to avoid relying on a single control. Trusted requests must match the expected source conditions, avoid rate-limit labels, avoid managed rule detections, and survive the default block posture.

4. Security Controls Implemented

Default-deny Web ACL

The Web ACL is associated with the API Gateway REST API and configured with a default Block action. Requests are denied unless an explicit allow rule later admits them.

Partner IP allow-list

A trusted partner NAT Gateway public IP is added to an AWS WAF IP set. Matching requests are counted and labeled with `ipallowed`.

Rate limiting

A rate-based rule blocks clients sending more than 50 requests within a one-minute evaluation window. Matching requests are labeled with `ratelimit`.

Compound allow rule

A custom allow rule admits only requests that have the `ipallowed` label and do not have the `ratelimit` label.

SQL injection protection

`AWSManagedRulesSQLiRuleSet` is added to detect SQL injection attempts in request components such as query parameters.

CloudWatch visibility

AWS WAF metrics and labels are reviewed in CloudWatch to understand allowed, blocked, counted and rate-limited traffic patterns.

5. Implementation Walkthrough

5.1 Baseline API Access

Before AWS WAF is associated with the API, a request to the inventory API returns product data. This confirms that the application path is functional before applying controls.

```
curl https://<api-id>.execute-api.<region>.amazonaws.com/Dev?id=1
```

Expected response:

```
["Tennis Racket Pro Series", "145 units", "$45.99"]
```

5.2 Create a Web ACL with Default Block

The first control is a Regional AWS WAF Web ACL associated with the API Gateway REST API. The Web ACL is configured with a default Block action and sampled requests enabled.

```
AWS WAF & Shield
-> Web ACLs
-> Create web ACL
-> Resource type: Regional
-> Associated resource: Amazon API Gateway REST API
-> Default action: Block
-> Sampled requests: Enabled
```

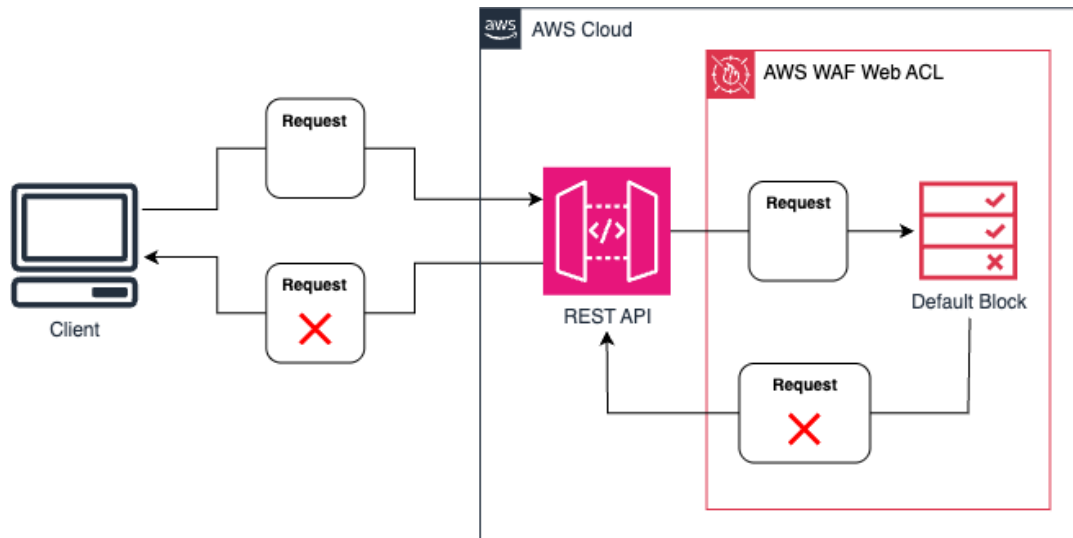


Figure 2 - Default-deny Web ACL behavior: requests that do not match an allow rule are blocked before reaching API Gateway.

5.3 Optional Geo Restriction

A geo-based rule was tested as an additional control to restrict requests by country. This is useful for coarse filtering, but it should not be treated as a primary trust mechanism because source geography can be manipulated through VPNs or proxy infrastructure.

BlockCpnnnections

< Manage rules for api-acl

Visual | JSON

Action

Block

Rule name

BlockCpnnnections

Names must use 1-128 characters from A-Z, a-z, 0-9, hyphen, and underscore.

If a request

matches the statement

Inspect

Originates from a country in

Statement

Country

Germany - DE

Figure 3 - Geo match rule used to test country-based request filtering in the Web ACL.

5.4 Partner IP Set and Labeling Rule

The next control is an IP set containing the partner network egress IP address in /32 CIDR format. This IP set is referenced by a rule in a custom rule group.

```
IP set name: anycompany_partners_ip_set  
Address: <partner-nat-public-ip>/32
```

```
Rule group: core-rule-group  
Rule: count_partner_request  
Action: Count  
Label: ipallowed
```

The screenshot displays the AWS IAM console interface for managing IP address sets. On the left, the 'Manage IP sets' section shows a list of IP address sets with one entry, 'anycompany_partners_ip_set', which is selected. The right pane shows the configuration details for this set. The 'Description' field is optional and empty. The 'Region' is set to 'US West (Oregon)'. The 'IP version' is set to 'IPv4'. The 'ARN' is displayed as 'arn:aws:wafv2:us-west-2:616258919473:regional/ipset/anycompany_partners_ip_set/10dd424a-e03a-461e-9341-be16208defef'. The 'IP addresses' field contains '44.224.241.245/32'. At the bottom, there are 'Cancel', 'Delete set', and 'Save' buttons.

Manage IP sets
An IP address set is a collection of IP addresses that can be reused across protection packs (web ACLs).

IP address sets (1/1)

Find IP sets

Region scope
Regional

Actions

Create IP address set

Name	Des...	ARN
anycompany_partners_ip_set		arn:aws:wafv2:us-west-2

anycompany_partners_ip_set

Description - optional
Enter description

Region
US West (Oregon)

IP version
IPv4

ARN
arn:aws:wafv2:us-west-2:616258919473:regional/ipset/anycompany_partners_ip_set/10dd424a-e03a-461e-9341-be16208defef

IP addresses
44.224.241.245/32
Enter IP addresses separated by commas or new lines in CIDR format.

Cancel Delete set Save

Figure 4 - Partner IP set creation using the NAT Gateway public IP in /32 format.

Add IP-based rule for Rule group
×

< Choose rule type for new rule group

Visual | JSON

Action

Count

Rule name

count_partner_request

Names must use 1-128 characters from A-Z, a-z, 0-9, hyphen, and underscore.

Statement

☒ Use existing IP set

IP address list

anycompany_partners_ip_set

▶ Rule configuration

▶ Custom request - optional

▼ Add labels - optional

Add labels to requests that match this rule. Rules that are evaluated later in the same protection pack (web ACL) can reference the labels that this rule adds.

Labels

Enter the string containing the label name and optional prefix and namespaces. For example, namespace1:name or awswaf:managed:aws:managed-rule-set:namespace1:name.

- Each namespace or name can have up to 128 characters
- You can specify up to 5 namespaces in a label
- Labels are case sensitive
- You can't use reserved names in labels. Reserved names include "awswaf", "aws", "waf", "rulegroup", "webacl", "regexpatternset", "ipset", and "managed".

Label

ipallowed

Remove

Add label

Cancel

Add rule

Figure 5 - IP-based rule that counts matching partner requests and applies the ipallowed label.

5.5 Rate-Based Blocking for Over-Requesters

To protect the API from abusive clients and burst traffic, a rate-based rule blocks clients that exceed the expected partner request volume.

```
Rule: block_overrequested
Type: Rate-based rule
Rate limit: 50 requests
Evaluation window: 1 minute
```

Action: Block
Label: ratelimit

The screenshot shows the 'Add rule-based rule for core-rule-group' interface in the AWS WAF console. At the top, there's a header 'Add rule-based rule for core-rule-group' and a back arrow with the text 'Choose rule type for core-rule-group'. Below this are two tabs: 'Visual' (selected) and 'JSON'. The 'Action' dropdown is set to 'Block'. The 'Rule name' field contains 'block_overrequested', with a note below it stating 'Names must use 1-120 characters from A-Z, a-z, 0-9, hyphen, and underscore.' The 'Statement' section has a 'Rate limit' of '50' and an 'Evaluation window' of '1 minute'. A note below the rate limit states 'Rate limit must be between 10 and 2,000,000,000. The default time span is 3 minutes.' There is a 'Rule configuration' section with a right-pointing arrow. Below that is a 'Custom response - optional' section with an 'Enable' checkbox. The 'Add labels - optional' section includes instructions on how to format labels and a list of reserved names. A 'Label' field contains 'ratelimit', with 'Add label' and 'Remove' buttons. At the bottom right are 'Cancel' and 'Add rule' buttons.

Figure 6 - Rate-based AWS WAF rule blocking clients that exceed 50 requests in one minute.

5.6 Allow Valid Partner Requests

Because the Web ACL default action is Block, an explicit allow rule is required. The allow rule uses label logic instead of relying directly on raw IP conditions.

```
Rule: allow_valid
Logic:
  has label: ipallowed
  AND NOT has label: ratelimit
Action: Allow
```

Rule order is critical. The label-producing rule group must run before allow_valid; otherwise the allow rule cannot evaluate the ipallowed and ratelimit labels.

allow_valid

×

< Manage rules for api-acl

Action

Allow

Rule name

allow_valid

Names must use 1-128 characters from A-Z, a-z, 0-9, hyphen, and underscore.

If a request

matches all the statement (AND)

Inspect

Has a label

Statement

☒ Label

☐ Namespace

Match key

Enter the string containing the label name and optional prefix and namespaces. For example, namespace1:name or aws:managed-rule-set:namespace1:name.

ipallowed

×

► Rule configuration

Remove

Inspect

Has a label

Statement

☒ Label

☐ Namespace

Match key

Enter the string containing the label name and optional prefix and namespaces. For example, namespace1:name or aws:managed-rule-set:namespace1:name.

ratelimit

×

► Rule configuration

Remove

Figure 7 - Compound allow rule requiring ipallowed and excluding ratelimit before allowing the request.

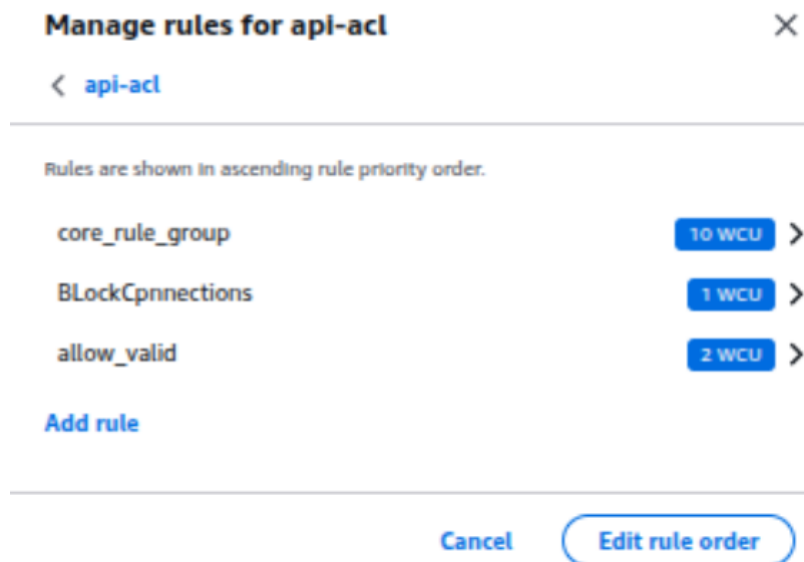


Figure 8 - Web ACL rule priority showing label-producing rules evaluated before the final allow rule.

5.7 Validate Access Control and Rate Limiting

Three validation tests were performed: non-partner access, partner access and high-volume partner traffic.

```
# Non-partner browser request
https://<api-id>.execute-api.<region>.amazonaws.com/Dev?id=1
Expected: {"message":"Forbidden"}

# Partner network request
curl https://$API_GATEWAY.execute-api.$AWS_REGION.amazonaws.com/Dev?id=1
Expected: ["Tennis Racket Pro Series", "145 units", "$45.99"]

# Rate-limit test
artillery run test.yml
Expected: initial 200 responses, followed by 403 responses once the rate limit is exceeded
```

```
sh-5.2$ sudo su - ec2-user
[ec2-user@ip-10-10-0-78 ~]$ curl https://$API_GATEWAY.execute-api.$AWS_REGION.amazonaws.com/Dev?id=1
[["Tennis Racket Pro Series","145 units","$45.99"]][ec2-user@ip-10-10-0-78 ~]$ pwd
```

Figure 9 - Partner network request allowed because the request originates from the trusted network path.

```
-----
Metrics for period to: 10:47:50(+0000) (width: 4s)
-----
http.codes.200: ..... 5
http.codes.500: ..... 4
http.downloaded_bytes: ..... 399
http.request_rate: ..... 3/sec
http.requests: ..... 13
http.response_time:
```

Figure 10 - Artillery traffic test showing successful requests followed by blocked responses after rate limiting is applied.

5.8 Block SQL Injection Attempts

The backend Lambda function builds a SQL query using the id query parameter. A malicious request can attempt to inject SQL syntax into the parameter to retrieve additional fields from the database.

```
curl "https://$API_GATEWAY.execute-api.$AWS_REGION.amazonaws.com/Dev?id=1%20UNION%20SELECT%20profit%20NULL%20warehouse%20FROM%20inventory%20WHERE%20id%3D1"
```

To add a WAF-layer mitigation, the AWS managed SQL database rule group is attached to the Web ACL.

Managed rule group: `AWSManagedRulesSQLiRuleSet`
Purpose: detect SQL injection patterns in request components
Action: Block

AWS-AWSManagedRulesSQLiRuleSet

< Manage rules for api-act

Details

Version
Default (using Version_1.3)

Capacity
200

Contains rules that allow you to block request patterns associated with exploitation of SQL databases, like SQL injection attacks. This can help prevent remote injection of unauthorized queries. [Learn more](#)

Inspection

☒ All requests ☐ Match statement

Rule overrides

☐ Override rule group
This changes the rule group action to Count mode but keeps all individual rule actions unchanged.

☒ Override all rule actions

SQLExtendedPatterns_QUERYARGUMENTS

SQL_QUERYARGUMENTS

SQL_BODY

SQL_COOKIE

SQL_URI_PATH

Available labels

☒ Subscribe to receive SNS notifications about new version.

☒ arn:aws:sns:us-east-1:248400274283:aws-managed-waf-rule-notifications

Cancel

Figure 11 - AWS managed SQLi rule group added to the Web ACL to detect SQL injection payloads.

Manage rules for api-act

< api-act

Rules are shown in ascending rule priority order.

core_rule_group 10 WCU >

AWS-AWSManagedRulesSQLiRuleSet 200 WCU >

allow_valid 2 WCU >

block-all 2 WCU >

[Add rule](#)

✓ Successfully updated the rule priority.

Figure 12 - Web ACL priority after adding AWSManagedRulesSQLiRuleSet before the final allow rule.

```
[ec2-user@ip-10-10-0-78 ~]$ curl https://$API_GATEWAY.execute-api.$AWS_REGION.amazonaws.com/Dev?id=1%20UNION%20SELECT%20profit%20NULL%20warehouse%20FROM%20inventory%20WHERE%20id%3D1  
{\"message\": \"Forbidden\"}
```

Figure 13 - SQL injection request blocked after enabling the AWS managed SQLi rule group.

6. Attack Validation and Detection Results

Test Case	Control Evaluated	Expected Result	Security Meaning
Browser request from non-partner IP	Default block + missing ipallowed label	403 Forbidden	Only trusted partner sources are allowed.
Request from partner NAT public IP	IP set + ipallowed label + allow_valid	200 OK	Expected partner traffic reaches the API.
High-volume partner traffic	Rate-based rule + ratelimit label	403 after threshold	Abuse is blocked even from a trusted source.
SQL injection payload in id parameter	AWSManagedRulesSQLiRuleSet	403 Forbidden	Injection payloads are blocked before the backend is reached.
Geo-restricted request	Geo match rule	Blocked when outside approved location	Coarse geographic filtering can reduce exposure, but is not identity-based.

7. Rule Evaluation Model

The most important implementation detail is rule ordering. Label rules must be evaluated before rules that rely on those labels.

Recommended evaluation model:

1. core-rule-group
 - count_partner_request -> adds ipallowed
 - block_overrequested -> adds ratelimit and blocks
2. Optional geo restriction
 - blocks requests outside approved country policy
3. AWSManagedRulesSQLiRuleSet
 - blocks SQL injection payloads
4. allow_valid
 - allows only ipallowed AND NOT ratelimit
5. Default Web ACL action
 - blocks everything else

This produces a practical default-deny model. A request must be from an approved partner source, must not exceed the rate threshold, must not match SQLi signatures, and must satisfy the final allow rule.

8. Monitoring and Operational Visibility

AWS WAF metrics and sampled requests are used to understand how the Web ACL behaves during normal traffic and attack simulations.

The following signals are useful for operational analysis:

- AllowedRequests and BlockedRequests by Web ACL
- CountedRequests for ipallowed label

- BlockedRequests for ratelimit label
- Top matching rules in the Web ACL traffic overview
- Attack type classification such as SQLi and volumetric activity



Figure 14 - AWS WAF traffic overview showing spikes in allowed and blocked requests during validation tests.

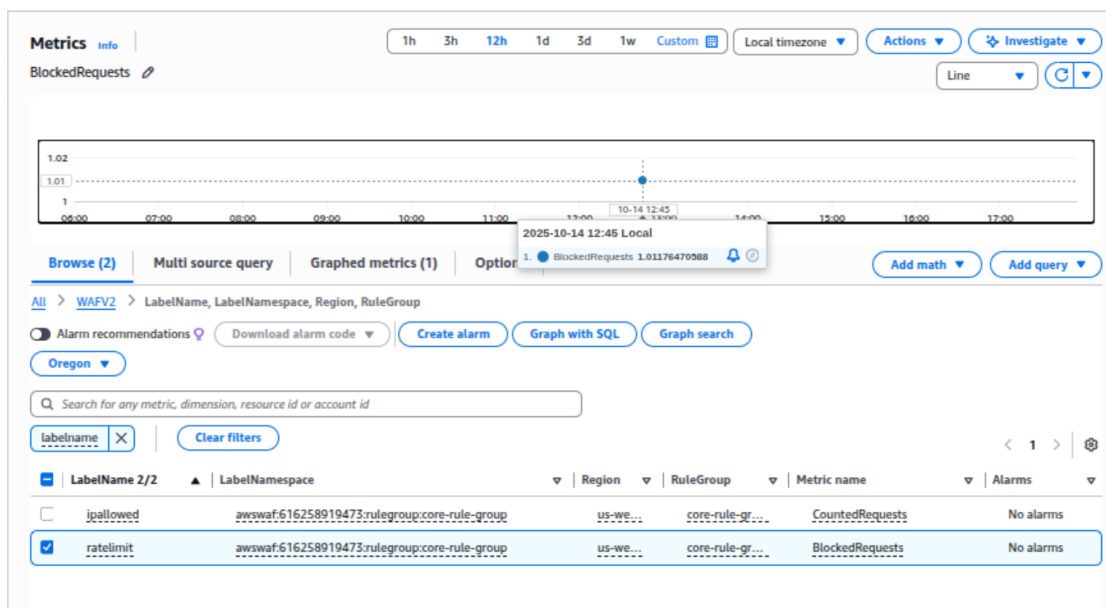


Figure 15 - CloudWatch WAFV2 metrics used to analyze labeled and blocked requests.

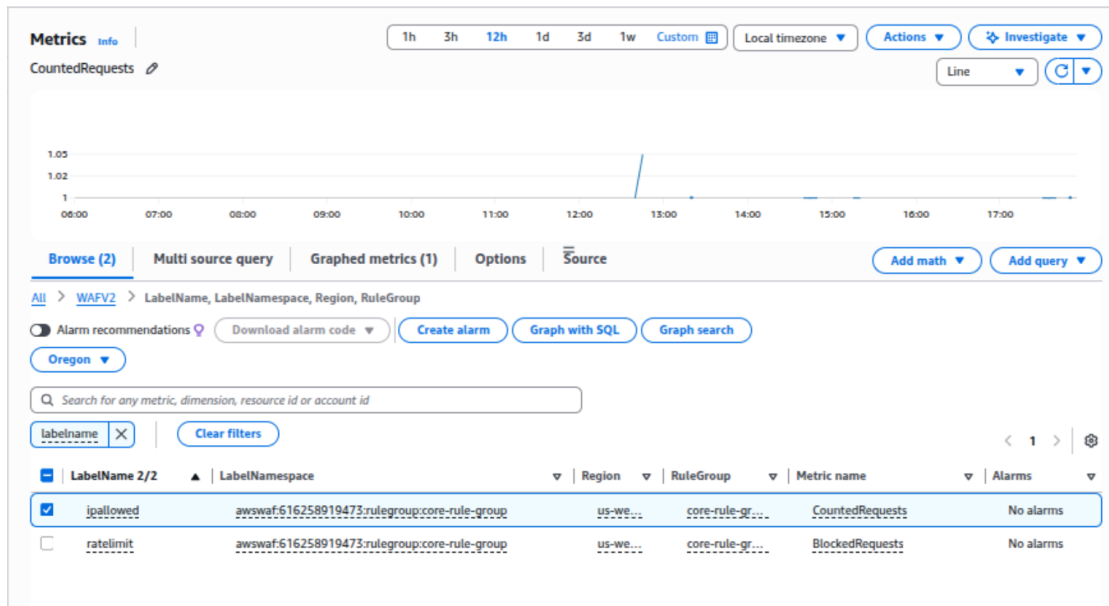


Figure 16 - CloudWatch metric view for request labels and WAF rule activity.

9. Security Lessons Learned

Default deny is the correct baseline

The Web ACL should block requests unless explicit allow criteria are met. This reduces accidental exposure.

Trusted sources can still behave maliciously

Rate limiting is still necessary even for partner networks, because valid clients can be compromised or misconfigured.

Labels improve rule composition

Using labels allows AWS WAF rules to classify requests first and make allow/block decisions later based on cumulative context.

Managed rules reduce exposure but do not replace secure coding

SQLi WAF rules are useful, but parameterized queries and input validation should still be implemented in the application.

Geo filtering is weak as a trust signal

Country-based blocking can reduce noise, but attackers can bypass it using VPNs or proxy infrastructure.

Monitoring is part of the control

A WAF without metrics and sampled request analysis is difficult to tune and validate.